4180/5180: Reinforcement Learning and Sequential Decision Making (Spring 2026) Robert Platt & Chris Amato
Northeastern University Mar. 19th, 2026

Exercise 6: Function Approximation

Please remember the following policies:

- Exercise due at **11:59 PM EST Mar 19, 2026**.
- Submissions should be made electronically on Canvas. Please ensure that your solutions for both the written and programming parts are present. You can upload multiple files in a single submission. Please do **not** zip them into a single file.
- You can make as many submissions as you wish, but only the latest one will be considered. Please do **not** make any submissions after the deadline unless you would like the submission to be considered as a late submission.
- For **Written** questions, solutions may be handwritten or typeset. If you write your answers by hand and submit images/scans of them, please please ensure legibility and order them correctly in a single PDF file.
- The PDF file should also include the figures from the **Plot** questions.
- For both **Plot** and **Code** questions, submit your source code in Jupyter Notebook (.ipynb file) along with reasonable comments of your implementation. Please make sure the code runs correctly. Please also submit a pdf of the Jupyter Notebook (.ipynb file).
- You are welcome to discuss these problems with other students in the class, but you must understand and write up the solution and code yourself. Also, you *must* list the names of all those (if any) with whom you discussed your answers at the top of your PDF solutions page.
- Each exercise may be handed in up to two days late (24-hour period), penalized by 10% per day late. Submissions later than two days will not be accepted.
- Contact the teaching staff if there are medical or other extenuating circumstances that we should be aware of.
- Generative AI can only be used for finding bugs or polishing English—all code and solutions should be written up by the student.
- **Notations: RL2e is short for the reinforcement learning book 2nd edition. x.x means the Exercise x.x in the book.**
- Please note that questions 3(b) and 3(e) are required for 5180 students and extra credit for 4180 students.
- Please note that questions 3(c) and 3(f) are extra credit questions for everyone.

1. **1 point.** (RL2e 10.1) *On-policy Monte-Carlo control with approximation.*
   Written: On the Mountain Car task, how do Monte Carlo methods perform relative to Temporal Difference methods?

2. **1 point.** (RL2e 10.2) *Semi-gradient expected SARSA and Q-learning.*
   Written:

   (a) Give pseudocode for semi-gradient one-step Expected Sarsa for control.

   (b) What changes to your pseudocode are necessary to derive semi-gradient Q-learning?

3. **4 points.** *Four rooms, yet again.*
   Let us once again re-visit our favorite domain, Four Rooms, as implemented in Ex4 Q3 (You can find it in the Notebook). We will first explore function approximation using *state aggregation*. Since this domain has four discrete actions with significantly different effects, we will only aggregate states; different actions will likely have different Q-values in the same state (or set of states).

   (a) **Code:** Implement semi-gradient one-step SARSA using the following state aggregation.

   - Each state is only aggregated to itself. In other words, the aggregated state space is **identical** to the original state space.
   - Since we use SARSA and want to learn Q values, we design the feature for each state(aggregated)-action pair using one-hot encoding. For example, if the size of the aggregate state space is N and the size of the action space is M, then given a state(aggregated)-action pair, you should represent it as a one-hot encoding vector $f = [0, 1, 0, .., 0]_d$, where $d = N \times M$.
   - Given the one-hot vector as the feature for each state(aggregated)-action pair, the weights are defined as a vector $w = [w_0, w_1, w_2, ..., w_d]$, where $d = N \times M$.
   - Following such design, we can approximate the Q value for one state(aggregated)-action pair as $Q(s, a) = w^T f(s, a) = w_i$, where $f(s, a)$ is the one-hot vector and i is the index of the state(aggregated)-action pair. Obviously, each element in w corresponds to the Q value of one state(aggregated)-action pair. We can call this strategy a *tabular* state aggregation.

   Verify that your implementation works by following the design above. Use the default hyperparameters in the Notebook.
   Plot: Plot learning curves with confidence bounds, as in past exercises. 100 episodes should be sufficient.
   Hint: To check whether your result is correct, you can compare your results against an implementation of tabular SARSA or Q-learning from Ex 5; the results should be similar.

   (b) **Code/plot: [5180 only]** Implement the following **two** state aggregation strategies, and plot the learning curves.

   - Tile-based state aggregation: We define a 2×2 tile and use it to aggregate the state space. For example, the state [0, 0], [1, 0], [0, 1], [1, 1] will be aggregated into one aggregated state. Importantly, the aggregated states are disjoint with each other.
   - Room-based state aggregation: We will aggregate the states in one Room into one aggregated state.

   Written: Comment on your findings, is aggregating the states helping to boost the learning speed? Shall we always aggregate the states as much as possible to learn better policy?

   (c) **[Extra credit.] 1 point.** Implement the semi-gradient one-step Q-learning and resolve (a) and (b) questions above. Comment on whether there is a difference between the results of semi-gradient SARSA and semi-gradient Q-learning?

We will now consider the more general case of linear function approximation.
If necessary, adapt your implementation of semi-gradient one-step SARSA for linear function approximation; this might not be necessary if your implementation is sufficiently general.

   (d) One natural idea is to use the (x, y) coordinates of the agent's location as features.
   Specifically, use the following three features for state s:

   - The state's x coordinate
   - The state's y coordinate

Problem 1.

- In the mountain car task, I think TD methods will outperform MC methods Because since we are going up to hill our velocity decreases thus our rewards time changes and timeline as well so having intermediate values would make the model more accurate instead of waiting for end of the episode.
So in summary TD methods update value function in episode which in this case very useful while MC methods just update end of episode.

Problem 2)

a) Psuedo code

Loop for each episode

init and Store $S_0 \neq$ terminal

$T \leftarrow 1$

Loop for $t = 0$ to $T-1$:

take action a from given state S

using ε - Greedy policy derived from

$q(S, \cdot, w)$

if S_{t+1} = terminal:

$G \leftarrow R$

else:

Select and compute

$G \leftarrow R + \gamma \sum \pi(a|S_{t+1}) \hat{q}(S_{t+1}, a, w)$

$w \leftarrow w + \alpha [G - \hat{q}(S, a, w)] \nabla \hat{q}(S, a, w)$

$S \leftarrow S_{t+1}$

until S is terminal

b) What changes for Semi-Gradient

Q-learning is that instead of $\pi(a|S')$ we will use maximization in next step $\max_a Q(S', a', w)$ So we will remove greedy part of SARSA $\sum \pi(a|S)$ and use $\max_a Q(S', a, w)$

- 1 (i.e., the feature value is 1 for any state)

Code/plot: Use these features for linear function approximation, and plot the learning curves.
Written: Why is the constant feature necessary? What do you think happens without it?
Also describe how you incorporated actions into your features.
Written: How do your results with the above features compare with state aggregation?
If there is a significant performance difference, try to come up with explanations for it.

(e) **Code/plot: [5180 only]** Implement the following **two** features, and plot the learning curves.

- Normalized state coordinates: use the feature $[x/10, y/10, 1]$
- Include distance: use the feature $[d, x, y, 1]$, where d is the shortest distance between the current state to the goal state. (You can find the distance matrix in the Notebook.)

Written: Comment on your findings, including any trends and surprising results.

(f) **[Extra credit.] 1 point.** One of the main benefits of function approximation is that it can handle large state spaces. So far, the Four Rooms domain is still quite small.
Design successively larger versions of Four Rooms, where each grid cell in the original environment can be subdivided into k cells per side (i.e., the number of states expands by a factor of k^2). You can choose whether or not to expand the walls/doorways.
Experiment with the various state aggregation and linear features proposed above (or propose more).
Plot your learning curves, and comment on your findings.